A Mini Project Report on

Crop Recommendation and Disease Detection System

B.E.- I.T Engineering

Submitted

by

Pranav Sathe (23104055)
Nayan Pingat (23104148)
Darshan Sanap (23104113)
Kavya Prajapati (23104178)

Under The Guidance of
Prof. Jayshree Jha



Department of Information Technology
A.P. Shah Institute of Technology
G.B.Road,Kasarvadavli, Thane(W), Mumbai-400615
UNIVERSITY OF MUMBAI
2026-2027

CERTIFICATE

This is to certify that the project Project entitled ***"Crop Recommendation and Disease Detection System"*** Submitted by ***"Pranav Sathe (23104055),Nayan Pingat (23104148),Darshan Sanap (23104113),Kavya Prajapati (23104178)"*** for the partial fulfillment of the requirement for award of a degree ***Bachelor of Engineering*** in ***Information Technology*** to the University of Mumbai,is a bonafide work carried out during academic year 2026-2027

Prof. Jayshree Jha
Guide

Dr.Kiran Deshpande
HoD,Information Technology

Dr. Uttam D.Kolekar
Principal

External Examiner(s)

1.

Place:A.P.Shah Institute of Technology, Thane
Date:

Declaration

I declare that this written submission represents my ideas in my own words and where others' ideas or words have been included, I have adequately cited and referenced the original sources. I also declare that I have adhered to all principles of academic honesty and integrity and have not misrepresented or fabricated or falsified any idea/data/fact/source in my submission. I understand that any violation of the above will be cause for disciplinary action by the Institute and can also evoke penal action from the sources which have thus not been properly cited or from whom proper permission has not been taken when needed.

(Pranav Sathe 23104055)
(Nayan Pingat 23104148)
(Darshan Sanap 23104113)
(Kavya Prajapati 23104178)

Date:

Acknowledgement

This project would not have come to fruition without the invaluable help of our guide Jayshree Jha. Expressing gratitude towards our HoD, Dr. Kiran Deshpande, and the Department of Information Technology for providing us with the opportunity as well as the support required to pursue this project. We would also like to thank our teacher Mr. Vishal Badgujar who gave us her valuable suggestions and ideas when we were in need of them. We would also like to thank our peers for their helpful suggestions.

Abstract

Crop selection and timely disease diagnosis are two of the most critical decisions in farming, yet many farmers still rely on experience and guesswork. This project presents a Flask-based web application that combines two machine learning capabilities to support data-driven decisions, with a focus on the Kokan region of Maharashtra. The crop recommendation module uses a Random Forest classifier on ten features (soil N, P, K, temperature, humidity, pH, rainfall, and three engineered indicators) to predict one of 22 crops, achieving 99.45% accuracy under 5-fold cross-validation. The disease detection module uses a Keras/TensorFlow CNN to classify leaf images into 31 disease categories across six crops (banana, cashew, coconut, mango, rice, sugarcane) and returns a remedy suggestion with each diagnosis. Region-aware logic tailors recommendations to five Kokan sub-regions, and a rule-based weather module, currently driven by representative regional data, flags disease risk before symptoms appear. To handle diseases outside its training set, the system flags predictions below 60% confidence as "Unrecognized Condition" and logs them to an admin review queue with user-suggested labels, forming a human-in-the-loop pipeline for future retraining. The application also provides authentication, a dashboard, and an SQLite backend.

Index

Contents

1 Introduction **6**

2 Objectives **8**

3 Literature Review **9**
3.1 Machine Learning for Crop Recommendation 9
3.2 Smart Agriculture Systems for Crop Selection 9
3.3 Random Forests in Agricultural Prediction 9
3.4 Research Gap 9
3.5 Comparison of Research Work 11

4 Problem Definition **12**
4.1 Problem Statement 12
4.2 Inputs, Outputs and Constraints 13

5 Proposed System Architecture/Working **14**
5.1 System Layers 14
5.2 Working of the System 15
5.3 Cost-Aware Behaviour 16

6 Design and Implementation **17**

7 Contribution **18**

8 Conclusion **19**

9 Future Scope **20**

References **21**

1 Introduction

Crop selection and disease diagnosis are two of the most important decisions in farming. Choosing a crop that does not suit the soil and climate leads to poor yield and financial loss, and a disease that is not identified early can destroy an entire harvest. Many farmers, especially in regions with varied micro-climates such as the Kokan coastal belt of Maharashtra, still rely on experience and guesswork because data-driven tools are not easily available to them.

Advances in Machine Learning (ML) and Deep Learning (DL) make it possible to build decision-support systems that recommend a suitable crop from soil and weather parameters and diagnose plant diseases from a simple leaf photograph. However, most such systems have two practical limitations. First, crop recommendation and disease detection are usually built as separate tools, and recommendations are generic rather than tailored to a region. Second, an image classifier is forced to assign every photo to one of its known classes, even when the disease is new, the crop is unsupported, or the photo is not of a leaf. The system then gives a confident but wrong diagnosis, and the error goes unnoticed.

This project presents a Crop Recommendation and Disease Detection System, a full-stack Flask web application that combines both capabilities and handles the second limitation explicitly. When the confidence of the disease classifier is low, the system does not show a diagnosis. It flags the case as an "Unrecognized Condition" and sends it to a human review queue.

The system is built from the following components:

- **Crop recommendation module:** A Random Forest classifier trained on soil nutrients (N, P, K), climate parameters (temperature, humidity, rainfall, pH) and three engineered indicators (NPK ratio, temperature-humidity index, soil fertility score) to predict one of 22 crops. Region-aware logic then refines the recommendation for five Kokan sub-regions.
- **Disease detection module:** A Keras/TensorFlow Convolutional Neural Network that classifies a leaf image into one of 31 disease categories across six crops (banana, cashew, coconut, mango, rice and sugarcane) and returns a treatment suggestion with each diagnosis.
- **Weather risk module:** A rule-based component that uses humidity, temperature and rainfall thresholds to classify disease risk as high, moderate or low, warning users before symptoms appear.
- **Unknown-disease feedback loop:** Predictions below 60% confidence are logged with the image and the model's rejected guess. Users can suggest what the condition might be, and an administrator reviews each case from a dedicated dashboard.
- **Web application:** A Flask backend with user authentication, a dashboard and an SQLite database, with HTML, CSS and JavaScript (Jinja2) templates for the interface.

It is important to state the scope of the work. The system is a proof-of-concept rather than a production deployment. The weather data is currently representative rather than fetched from a live weather service, and the review queue collects unrecognized cases for manual, offline retraining rather than retraining the model automatically. The contribution of the project lies

in combining the two models into a region-aware application, and in the confidence-threshold and human-in-the-loop design for handling diseases the model has not seen.

To evaluate the system, the Random Forest model is assessed using 5-fold stratified cross-validation, and the complete application is verified through functional test cases covering authentication, input validation, prediction, access control and the unknown-disease workflow.

2 Objectives

The main objective of this project is to design and implement a web-based decision-support system that combines crop recommendation and crop disease detection for farmers in the Kokan region of Maharashtra, and that handles diseases outside its training set by flagging low-confidence predictions for human review.

- To design and train a Random Forest model that recommends the most suitable crop from soil nutrients (N, P, K), climatic factors (temperature, humidity, rainfall, pH) and three engineered indicators (NPK ratio, temperature-humidity index, soil fertility score).
- To build and integrate a Keras/TensorFlow CNN that classifies leaf images into 31 disease categories across six crops (banana, cashew, coconut, mango, rice and sugarcane).
- To return a practical treatment or prevention suggestion with every confident diagnosis, rather than only naming the disease.
- To provide region-aware recommendations for five Kokan sub-regions (Ratnagiri, Sindhudurg, Mumbai, Thane and Palghar) using local soil type and cropping patterns.
- To implement a rule-based weather module that classifies disease risk as high, moderate or low from humidity, temperature and rainfall, so that users are warned before symptoms appear.
- To apply a 60% confidence threshold to the CNN so that uncertain predictions are shown as "Unrecognized Condition" instead of a possibly wrong diagnosis.
- To build a human-in-the-loop feedback pipeline in which unrecognized cases are logged, users can suggest what the condition might be, and an administrator reviews each case from a dedicated dashboard, so that the data can support future retraining.
- To build a Flask web application with user authentication, a dashboard, a profile page and an SQLite database, with input validation on all crop-recommendation parameters.
- To evaluate the Random Forest model using 5-fold stratified cross-validation and to verify the complete application through functional test cases covering authentication, validation, prediction, access control and the unknown-disease workflow.

3 Literature Review

This section reviews prior work in the areas that this project builds on: machine learning for crop recommendation, smart agriculture systems for crop selection, and the use of Random Forests in agricultural prediction.

3.1 Machine Learning for Crop Recommendation

Doshi et al. proposed AgroConsultant [1], a crop recommendation system that applies machine learning algorithms to soil and environmental data, so that crop selection depends less on intuition and traditional practice. Pande et al. presented a crop recommender based on a machine learning approach [2], using algorithms such as Random Forest on soil and climate data to improve prediction accuracy. Both works show that soil nutrients and climate parameters can be turned into data-driven crop recommendations. However, they recommend a crop from general soil and weather inputs. They are not described as adapting the advice to the soil type and cropping pattern of a specific region, and they do not include disease diagnosis.

3.2 Smart Agriculture Systems for Crop Selection

Namboodiri et al. introduced AGROFERDURE [3], a smart agriculture system that uses machine learning to help farmers select crops and make better decisions. Nandhini and Devi [4] likewise studied crop recommendation using machine learning. These works show that such systems can be offered to farmers as practical decision-support tools. However, they focus on the choice of crop before cultivation. They do not help a farmer once a crop is already growing and falls ill, and they do not warn of disease risk from weather conditions.

3.3 Random Forests in Agricultural Prediction

Jeong et al. used Random Forests for global and regional crop yield predictions [5], and Sujatha and Isakki studied crop yield prediction using machine learning [6]. A Random Forest builds many decision trees on random subsets of the data and features and combines their outputs by majority voting. It handles numerical inputs such as nitrogen, phosphorus, potassium, temperature, humidity and rainfall, it resists overfitting, and it reports which features matter most. These properties make it a suitable choice for crop recommendation. However, these studies work on tabular data only. They do not diagnose crop diseases from images, and they do not consider what a deployed model should do when an input is unlike its training data.

3.4 Research Gap

The surveyed work centres on selecting a crop or predicting yield from soil and climate data. Four needs are left open. First, recommendations are not tailored to a specific region such as the Kokan coastal belt. Second, crop selection and disease diagnosis are treated as separate problems, and a diagnosis is not accompanied by a practical remedy. Third, farmers are not warned of disease risk before symptoms appear. Fourth, a classifier is not given a way to

recognize that it is unsure, and unsure cases are not recorded for review. This project addresses these gaps by combining a Random Forest crop recommender with region-aware logic, a CNN disease detector that returns remedies, a rule-based weather risk module, and a confidence threshold with a human review queue for unrecognized conditions.

3.5 Comparison of Research Work

Comparison of research work			
Sr. No.	**Paper Title**	**Author Name**	**Key Findings**
1	Crop Recommender System Using Machine Learning Approach [2]	S. M. Pande et al.	Machine learning on soil and climate data to recommend crops, with Random Forest used to improve prediction accuracy.
2	AgroConsultant: Intelligent Crop Recommendation System Using Machine Learning Algorithms [1]	Z. Doshi et al.	Reduces dependence on intuition and traditional practice by predicting crops with machine learning.
3	AGROFERDURE: Intelligent Crop Recommendation System for Agriculture Crop Productivity Using Machine Learning Algorithm [3]	T. Namboodiri et al.	Smart agriculture system that uses machine learning to help farmers select crops and make decisions.
4	Crop Recommendation System Using Machine Learning [4]	K. R. Nandhini and S. A. S. Devi	Studies crop recommendation using machine learning techniques.
5	Random Forests for Global and Regional Crop Yield Predictions [5]	J. Jeong et al.	Applies Random Forests to crop yield prediction at global and regional scale.
6	A Study on Crop Yield Prediction Using Machine Learning [6]	R. Sujatha and P. Isakki	Studies the use of machine learning for crop yield prediction.
7	**Proposed system**	This project	Combines Random Forest crop recommendation with Kokan region logic, CNN disease detection with remedies, a weather risk module, and a 60% confidence threshold with human review.

Table 3.1: Comparison of research work

4 Problem Definition

Farmers in regions with varied micro-climates, such as the Kokan coastal belt of Maharashtra, must decide which crop to grow each season and how to respond when a crop falls ill. Existing tools usually address only one of these needs, and they rely on generic, one-size-fits-all advice. Even when a machine learning model is used, a deployed system rarely checks how reliable its own output is. This design leads to the following problems:

1. **Generic crop advice:** Crop recommendation tools are often built for a country-wide dataset and do not account for local soil type, cropping patterns or high-rainfall conditions. A crop suggested for the wrong region leads to poor yield, wasted resources and financial loss.

2. **Separate tools for separate needs:** Crop selection and disease diagnosis are usually offered as different applications, so a farmer must switch between tools, and neither tool uses the context of the other.

3. **Forced wrong diagnoses:** An image classifier must assign every photo to one of its known classes. When the disease is new, the crop is unsupported or the photo is poor, the model still returns a diagnosis, and the farmer may apply the wrong treatment.

4. **Reactive, not proactive, disease handling:** Most systems diagnose a disease only after symptoms are visible. They do not use humidity, temperature and rainfall to warn farmers when conditions favour known diseases, by which time damage may already have started.

5. **No feedback path for unknown cases:** Even when a system detects that it is unsure, there is usually no mechanism to record the case, collect what the user believes the condition is, and let an expert review it, so the system stays frozen at its original training set.

6. **Diagnosis without action:** Naming a disease is not enough for a farmer. Without a practical remedy or prevention step, the diagnosis cannot be acted on.

This project addresses these problems with a region-aware crop recommendation and disease detection system that flags low-confidence predictions as "Unrecognized Condition" and routes them to a human review queue.

4.1 Problem Statement

Farmers in regions with varied micro-climates, such as the Kokan coastal belt of Maharashtra, often lack easy access to data-driven tools that recommend the right crop for their soil and weather, or quickly diagnose a disease affecting their crop. Existing tools tend to give generic advice, treat crop selection and disease diagnosis as separate problems, and diagnose only after symptoms are visible. In addition, an image classifier must assign every photo to one of its known classes, so a new disease or a poor photo can produce a confident but wrong diagnosis,

and there is usually no way to record such cases for expert review. This project addresses these gaps with a region-aware crop recommendation and disease detection system that suggests practical remedies, warns of disease risk from weather conditions, and flags low-confidence predictions as "Unrecognized Condition" for human review.

4.2 Inputs, Outputs and Constraints

- **Input:** For crop recommendation, soil and climate values (N, P, K, temperature, humidity, pH and rainfall) together with the user's selected Kokan region. For disease detection, a JPG or PNG photograph of a plant leaf.
- **Output:** For crop recommendation, the predicted crop (one of 22) with a region-based message. For disease detection, the predicted crop and disease with a confidence percentage and a remedy suggestion, or an "Unrecognized Condition" result when confidence is low. A weather-based disease risk level (high, moderate or low) with likely diseases and advice is also shown, and every unrecognized case is logged for admin review.
- **Constraints:** Soil and climate inputs must lie within the validated ranges (for example, pH between 1 and 14 and rainfall up to 4500 mm). Uploaded images must be JPG or PNG and are resized to 224×224 pixels. Predictions below 60% confidence are never shown as a diagnosis. The CNN can only name one of its 31 trained disease classes, and crop recommendation is limited to 22 crops. Weather data is currently representative rather than live, and only registered, logged-in users can make predictions. The admin dashboard is restricted to accounts with the admin flag.

5 Proposed System Architecture/Working

The system is organised as a layered web application, shown in Figure 1. A registered user submits soil and climate values or a leaf image through a web interface. The Flask application validates the input, passes it to the crop recommendation, disease detection or weather risk module, loads the saved model files, and returns the result to the browser. Predictions below 60% confidence are not shown as a diagnosis and are logged for administrator review.

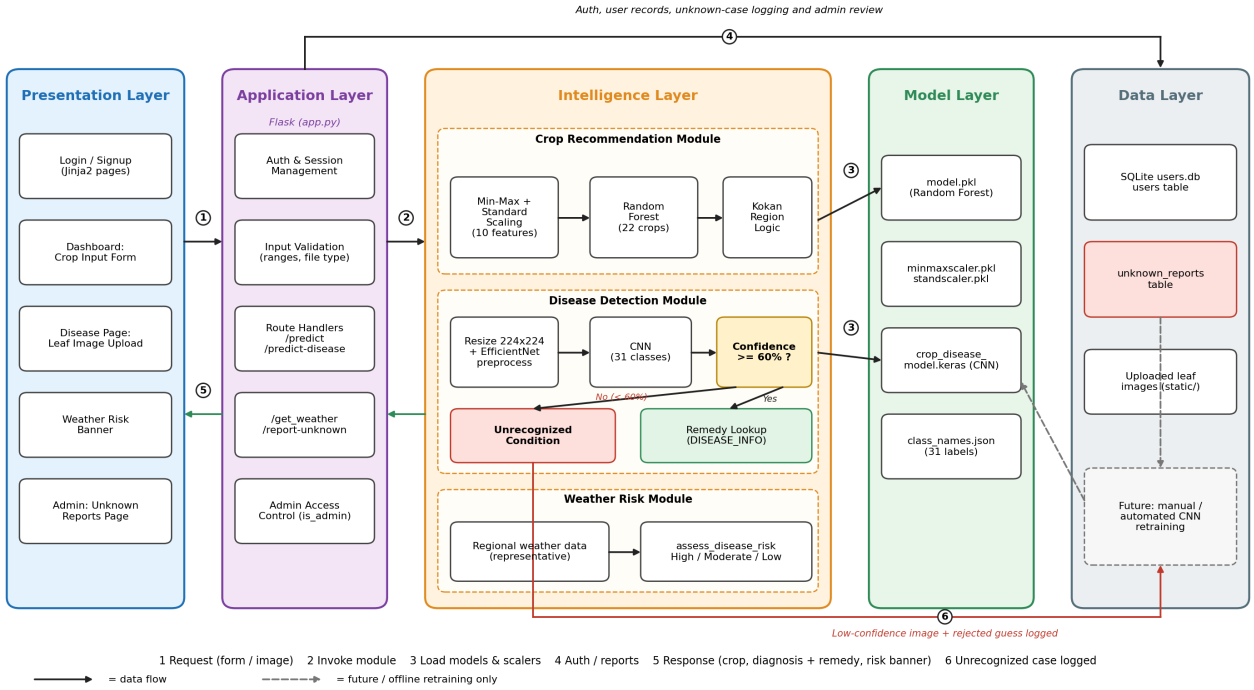


Figure 1: Proposed system architecture of ConfidenceRoute

5.1 System Layers

- **Presentation layer:** HTML, CSS and JavaScript pages rendered with Jinja2 templates: login and signup, a dashboard with the crop input form and weather risk banner, a disease detection upload page, a profile page and an admin page for unknown-disease reports.
- **Application layer:** A Flask application (`app.py`) that handles authentication and sessions, validates inputs (value ranges and JPG/PNG file type), and exposes the routes `/predict`, `/predict-disease`, `/get_weather/<region>`, `/report-unknown` and the admin routes. Admin pages are restricted to accounts with the `is_admin` flag.
- **Intelligence layer:** Three modules. The crop recommendation module scales the ten input features and predicts one of 22 crops with a Random Forest, then applies Kokan region logic to the result. The disease detection module preprocesses the leaf image, classifies it with a CNN into one of 31 classes, and applies the confidence threshold.

The weather risk module classifies disease risk as high, moderate or low from humidity, temperature and rainfall.

- **Model layer:** Saved model artifacts loaded by the modules: `model.pkl` (Random Forest), `minmaxscaler.pkl` and `standscaler.pkl` (feature scalers), `crop_disease_model.keras` (CNN) and `class_names.json` (the 31 disease labels).
- **Data layer:** An SQLite database (`users.db`) with a `users` table and an `unknown_reports` table, together with the uploaded leaf images stored on disk.

5.2 Working of the System

The system processes a request through the following steps.

1. **Authentication:** The user signs up or logs in. Passwords are hashed before storage, and a session (with an optional 30-day "remember me" cookie) gives access to the protected pages.
2. **Request submission:** For crop recommendation, the user enters N, P, K, temperature, humidity, pH and rainfall and selects a Kokan region. For disease detection, the user uploads a JPG or PNG leaf photograph.
3. **Validation:** The application checks that every value lies within its allowed range, that a region is selected, and that the uploaded file has a valid type. Invalid requests are rejected with an error message.
4. **Crop recommendation:** The three engineered features (NPK ratio, temperature-humidity index and soil fertility score) are computed, and the ten features are scaled with the saved Min-Max and Standard scalers. The Random Forest predicts a crop, and the region logic checks it against the region's preferred crops to confirm the choice or suggest a better-adapted alternative.
5. **Disease detection:** The image is converted to RGB, resized to 224×224 and normalised with EfficientNet preprocessing. The CNN outputs a probability for each of the 31 classes, and the most probable class and its confidence are taken.
6. **Confidence decision:** If the confidence is at or above 60%, the class name is split into crop and disease, and a remedy is looked up from the curated remedy table. If it is below 60%, no disease name is shown, and the user sees an "Unrecognized Condition" message instead.
7. **Weather risk assessment:** For the selected region, the weather module checks humidity, temperature and rainfall against its rules and returns a risk level with the likely diseases and an advisory message, shown as a banner on the dashboard and disease detection pages.

8. **Unknown-case logging:** For an unrecognized result, the image path, the model's rejected best guess and its confidence are saved in the `unknown_reports` table. The user may optionally submit what they believe the condition is.

9. **Administrator review:** An administrator opens the review dashboard, sees each flagged image with the model's guess and the user's suggestion, and marks the case as reviewed.

10. **Response:** The recommendation, diagnosis with confidence and remedy, risk banner, or unrecognized-condition message is returned to the browser.

5.3 Cost-Aware Behaviour

The system is designed to keep operating cost low. Both models run locally from saved files (`model.pkl` and `crop_disease_model.keras`), so no paid prediction service is called, the weather module uses representative regional data and needs no external API, and user accounts and flagged reports are stored in a lightweight SQLite database that runs on modest hardware (an Intel i3 processor and 4 GB of RAM at minimum). Cost is also controlled through confidence: a prediction at or above 60% is shown as a diagnosis in a single pass, while a prediction below 60% is shown as "Unrecognized Condition" and logged, so administrator effort is spent only on the cases the model is unsure about. The system does not choose between models based on cost, since it uses a single Random Forest and a single CNN.

6 Design and Implementation

Design of the working project model,code and project snapshots with detailed explanation to be included.

7 Contribution

This project, the Crop Recommendation and Disease Detection System, has been developed as an academic mini project (ROSPL), and its source code is maintained in a project repository.

- **Source code:** The complete application, including the Flask backend, trained model files, templates and test scripts, is available in the project repository at [add repository URL].
- **Development scope:** The system was built as a working proof-of-concept, with the crop recommendation module, disease detection module, weather risk module, authentication and the unknown-disease review workflow implemented and verified through functional test cases.

The original contributions of this work are:

- A combined web application that integrates a Random Forest crop recommendation model and a CNN disease detection model, with user authentication, a dashboard and an SQLite backend.
- Three engineered agronomic features (NPK ratio, temperature-humidity index and soil fertility score) added to the crop recommendation dataset, giving ten input features.
- Region-aware recommendation logic for five Kokan sub-regions (Ratnagiri, Sindhudurg, Mumbai, Thane and Palghar), based on local soil type and cropping patterns.
- A rule-based weather module that classifies disease risk as high, moderate or low from humidity, temperature and rainfall, so that users are warned before symptoms appear.
- A 60% confidence threshold on the CNN that flags uncertain predictions as "Unrecognized Condition" instead of showing a possibly wrong diagnosis.
- A human-in-the-loop feedback pipeline in which unrecognized cases are logged, users can suggest what the condition might be, and an administrator reviews each case from a dedicated dashboard.
- A curated remedy table that maps each disease class to a practical treatment or prevention suggestion.

Existing open-source tools such as Flask, scikit-learn, TensorFlow/Keras, Pandas, NumPy, Pillow and SQLite are used under their own licenses. The Random Forest model is trained on a publicly available crop recommendation dataset, and the contribution of this work lies in integrating these tools into one region-aware application rather than in developing new machine learning libraries.

8 Conclusion

This project presented ConfidenceRoute, an adaptive orchestration layer for multi-agent LLM systems. Existing frameworks fix the sequence of steps and the model for each step in advance, which wastes money on easy sub-tasks and lets weak outputs pass downstream unnoticed. ConfidenceRoute addresses both problems by scoring the output of every agent step at runtime and using that score to decide whether to proceed, retry, escalate to a stronger model, or insert a verification step.

The work produced the following results:

- A ReAct reasoning loop was implemented from scratch, with its own prompt construction, action parsing and tool dispatch, and it runs on a locally hosted Llama 3 (8B) model.
- A Random Forest confidence classifier was trained on the HaluEval dataset using features engineered from response text. On a held-out test set it achieved an accuracy of [**A**]% and an F1-score of [**F**], compared with 50% for random guessing on the balanced dataset.
- An adaptive orchestrator was built that turns the classifier's score into routing decisions, with a bounded number of retries so that execution always terminates.
- A FastAPI and WebSocket backend with a React dashboard was developed, so that each step, its confidence score and the model used can be watched live.
- On [**N**] HotpotQA questions, the adaptive system achieved a success rate of [**X**]%, compared with [**Y**]% for a static baseline. It used the stronger model on [**Z**]% of questions and made an average of [**R**] retries per question.

These results show that a lightweight, explainable classifier can provide a useful runtime quality signal, and that using such a signal to control retries and escalation is a practical way to balance accuracy and cost in multi-agent pipelines. The main advantage of the approach is that cheap local models handle the default case, while expensive models are reserved for outputs that the classifier flags as unreliable.

The work also has limitations. The classifier was trained on HaluEval, whose hallucinated answers were generated by ChatGPT, so its performance on Llama 3 outputs may differ from its test accuracy. The features are surface-level properties of the text, so the classifier estimates how reliable an answer looks and cannot verify facts. The benchmark used a small sample of questions and a single dataset, so the findings should be read as evidence for the approach and not as a general guarantee. These limitations motivate the extensions described in the next section.

9 Future Scope

The project can be extended in the following ways:

- **Live weather integration:** A real weather service (for example, OpenWeatherMap with an API key) can replace the representative regional data, so that crop recommendations and disease risk warnings use real-time conditions.
- **Automated retraining pipeline:** Admin-approved entries from the unknown-disease review queue can be added to the CNN's training set periodically, extending it to recognize new disease classes without manual, offline retraining.
- **Stronger unknown-disease detection:** The fixed 60% threshold can be improved with calibrated confidence, entropy-based scores or an extra "not a leaf" class, so that non-leaf images and unseen diseases are caught more reliably.
- **Full prediction history:** Predictions can be stored and retrieved per user, and the data export and account deletion features, currently placeholders, can be completed.
- **Yield and market-price prediction:** The system can estimate potential yield and farmer profit alongside the crop recommendation.
- **Multilingual and voice support:** Hindi and Marathi interfaces and voice input can make the system accessible to more farmers.
- **Mobile and offline use:** A mobile application with a lightweight, offline-capable model can serve areas with poor connectivity.
- **Broader and validated rules:** The disease-risk rule set can be expanded with more region- and crop-specific thresholds validated against agricultural extension data, and the models can be evaluated on real field data from the Kokan region.

References

[1] Z. Doshi et al., "AgroConsultant: Intelligent Crop Recommendation System Using Machine Learning Algorithms," 2018.

[2] S. M. Pande et al., "Crop Recommender System Using Machine Learning Approach," 2021.

[3] T. Namboodiri et al., "AGROFERDURE: Intelligent Crop Recommendation System for Agriculture Crop Productivity Using Machine Learning Algorithm," [year].

[4] K. R. Nandhini and S. A. S. Devi, "Crop Recommendation System Using Machine Learning," 2019.

[5] J. Jeong et al., "Random Forests for Global and Regional Crop Yield Predictions," 2016.

[6] R. Sujatha and P. Isakki, "A Study on Crop Yield Prediction Using Machine Learning," 2020.